



CompSci 230

Object Oriented Software Development

Control Flow



Review Quizzes

- ▶ What is the difference between the following two statements? If x is 1, what is the value of x after each statement?

- ▶ `x += 10;` //

- ▶ `x =+ 10;` equal

- ▶ What will be the value of z as a result of executing the following code?

```
int x = 5, y = 28;  
float z;  
z = (float) (y / x);
```

- ▶ 5.60

- ▶ 5.6

- ▶ 3.0

- ▶ 5.0 ✓



Today's Agenda

► Topics:

- Repetition structure
 - Sentinel-Controlled Repetition
 - Counter-Controlled Repetition
 - The for, while, do-while statements
- Branching Statements
 - The break and continue Statements, the return statement, exit
- Random numbers

► Reading:

- Java how to program Late objects version (D & D)
 - Chapter 3 & Chapter 4
- The Java Tutorials
 - Lesson: Language Basics: Control Flow Statements
 - <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html>
- Welcome to Java for Python Programmers
 - <http://interactivepython.org/courselib/static/java4python/index.html>



1.Repetition Structure

- ▶ It is also called iteration statements or looping statements
- ▶ Perform statements repeatedly while a **loop-continuation condition** remains true.
- ▶ **while** and **for** statements perform the action(s) in their bodies **zero** or **more** times
 - ▶ if the loop-continuation condition is initially **false**, the body will **not** execute.
- ▶ The **do...while** statement performs the action(s) in its body **one** or **more** times.



1.Repetition Structure

The while Statement

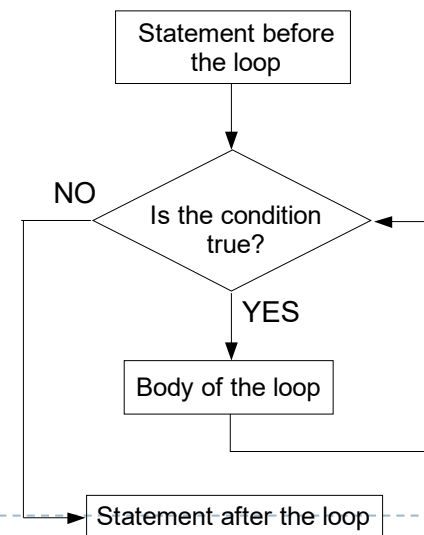
- ▶ The while repetition statement
 - ▶ repeats an action while a condition remains true.
- ▶ The repetition statement's body may be a single statement or a block.
- ▶ Eventually, the condition will become false. At this point, the repetition terminates, and the first statement after the repetition statement executes.

```
statementsBeforeLoop;  
while( condition ) {  
    statement_1;  
    ...  
    statement_n;  
}  
statementsAfterLoop;
```

▶ Example:

```
int numberOfCoins = 4;  
while( numberOfCoins > 0 ) {  
    numberOfCoins--;  
    System.out.println("Insert a coin");  
}
```

```
Insert a coin  
Insert a coin  
Insert a coin  
Insert a coin
```





1.Repetition Structure

Sentinel-Controlled Repetition

- ▶ It is often called **indefinite** repetition because the number of repetitions is **not** known before the loop begins executing.
- ▶ A special value called a **sentinel** value (also called a signal/dummy/flag value) can be used to indicate “end of data entry.”
- ▶ Program logic for sentinel-controlled repetition
 - ▶ Reads the **first** value before reaching the loop.
 - ▶ If the condition of the loop is **false**, so the body of the loop does not execute
 - ▶ If the condition is **true**, the body begins execution and processes the input.
 - ▶ Then the loop body **inputs** the next value from the user before the end of the loop.



1.Repetition Structure

Sentinel-Controlled Repetition

- ▶ Example: Develop a class-averaging program that processes grades for an arbitrary number of students each time it's run.
- ▶ Steps:
 - ▶ Initialize variables (initialize total, count to zero)
 - ▶ Input, sum and count the quiz grades
 - ▶ Calculate and print the class average
- ▶ For step 2: Input, sum and count the quiz grades
 - ▶ It requires a repetition to successively input each grade.
 - ▶ We do not know in advance how many grades will be entered, so we'll use sentinel-controlled repetition.

```
Prompt the user to enter the first grade  
Input the first grade (possibly the sentinel)  
while the user has not yet entered the sentinel  
    Add this grade into the running total  
    Add one to the grade counter  
    Prompt the user to enter the next grade
```



1.Repetition Structure Example

- ▶ Develop a class-averaging program that processes grades for an arbitrary number of students each time it's run.
 - ▶ sentinel-controlled: when grade is -1

```
Scanner input = new Scanner(System.in);  
int total = 0, gradeCounter = 0;  
  
System.out.print("Enter grade or -1 to quit: ");  
int grade = input.nextInt();  
  
while (grade != -1) {  
    total += grade;  
    gradeCounter += 1;  
    System.out.print("Enter grade or -1 to quit: ");  
    grade = input.nextInt();  
}
```

```
Enter grade or -1 to quit: 97  
Enter grade or -1 to quit: 88  
Enter grade or -1 to quit: 72  
Enter grade or -1 to quit: -1
```

```
Total of the 3 grades entered is 257  
Class average is 85.67
```

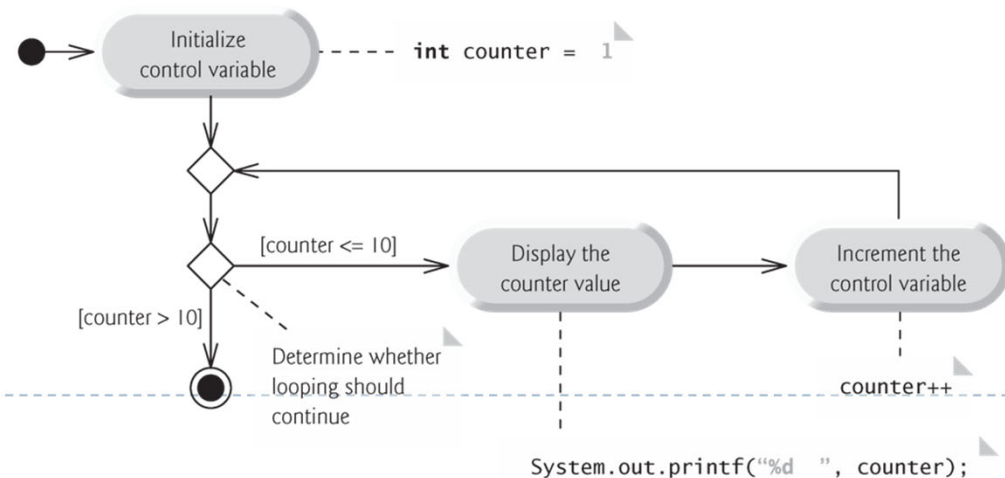



1.Repetition Structure

Counter-Controlled Repetition

(For number of iterations)

- ▶ A variable called a counter (or control variable) controls the **number of times** a set of statements will execute.
- ▶ It is often called **definite repetition**, because the number of repetitions is **known** before the loop begins executing
- ▶ Counter-controlled repetition requires
 - ▶ the **initial** value of the **control** variable
 - ▶ the **increment** by which the control variable is modified each time through the loop (also known as each iteration of the loop)
 - ▶ the **loop-continuation condition** that determines if looping should continue.

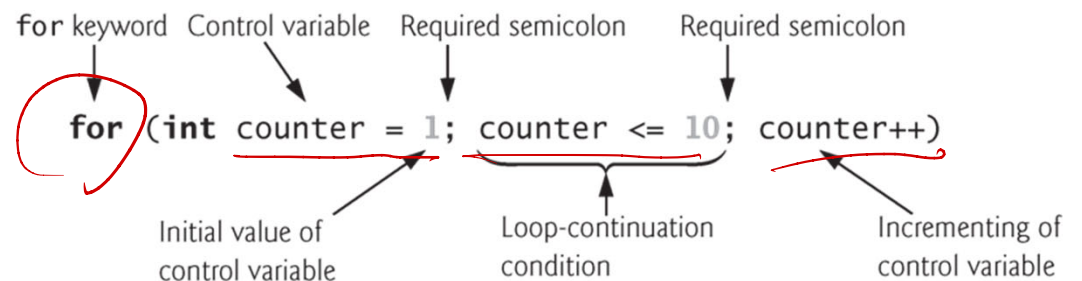




1.Repetition Structure

Counter-Controlled Repetition: for

- ▶ Specifies the counter-controlled-repetition in a single line of code
 - ▶ When the for statement begins executing, the control variable is **declared** and **initialized**.
 - ▶ Next, the program checks the **loop-continuation condition**, which is between the two required semicolons.
 - ▶ If the condition initially is **true**, the body statement **executes**.
 - ▶ After executing the loop's body, the program **increments** the **control** variable in the increment expression.
 - ▶ Then the loop-continuation test is **performed** again to determine whether the program should continue with the next iteration of the loop.



- ▶ The general form:

```
for (initialization; loopContinuationCondition; increment)
    statement
```

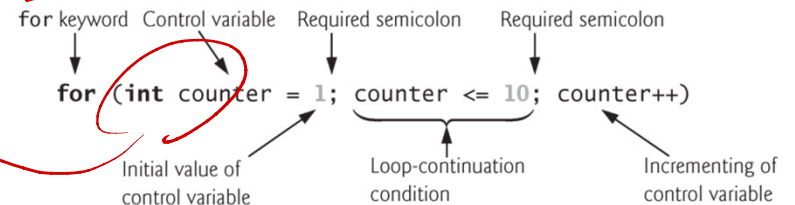


1.Repetition Structure

Counter-Controlled Repetition: for

- ▶ If the loop-continuation condition is initially **false**, the program does **not** execute the for statement's body.
- ▶ If the initialization expression in the for header **declares** the control variable, the control variable can be used only in that for statement.

Only be used inside the for loop.



- ▶ The two semicolons in the for header are required.
- ▶ All three expressions in a for header are optional. But if ...
 - ✱ loopContinuationCondition is omitted, thus creating an infinite loop.
 - ▶ initialization is omitted only if your program initializes the control variable before the loop.
- ▶ The initialization and increment can be comma-separated lists that enable you to use multiple initialization or multiple increment.
- ▶ The increment of a for statement may be negative, in which case it's a decrement, and the loop counts downward.



1.Repetition Structure Examples

- Initialization, loop-continuation condition, increment can be comma-separated lists of expressions
- Example 1

```
int number = 2, total=0;  
for(; number <= 20; total += number, number += 2)  
    ; // empty statement  
  
System.out.println("number=" + number);  
System.out.println("total=" + total);
```

loop-continuation
condition

number=22
total=110

still
required

but why?

- Example 2

```
total = 0;  
for (int n = 2; n <=20; n +=2 ) {  
    total += n;  
    System.out.println(n);  
}  
//System.out.println(n);  
System.out.println("total=" + total);
```

calculate total in
the body

total=110

"n" is only visible
within the loop



1.Repetition Structure

Counter-Controlled Repetition: while

- ▶ The *for* statement often can be represented with an equivalent *while* statement as follows:

```
initialization;
while (loopContinuationCondition) {
    statement
    increment;
}
```

```
total=0;
number=2;
while (number <= 20) {
    total += number;
    number +=2;
}
System.out.println("total=" + total);
```

A red arrow pointing from the while loop in the code block above to the for loop in the code block below.

- ▶ Typically, for statements are used for counter-controlled repetition and while statements for sentinel-controlled repetition.



1.Repetition Structure

The do-while Statements

- ▶ The do...while repetition statement is similar to the while statement.
- ▶ In the while, the program tests the loop-continuation condition at the beginning of the loop, before executing the loop's body; if the condition is false, the body never executes.
- ▶ The do...while statement tests the loop-continuation condition **after** executing the loop's body; therefore, the body always executes at least **ONCE**.
 - ▶ When a do...while statement terminates, execution continues with the next statement in sequence.

- ▶ Syntax:

```
do {  
    statement(s)  
} while (expression);
```



1.Repetition Structure Examples

1 2 3 4 5 6 7 8 9 10 END-11

► Example 1:

```
counter = 1;
do {
    System.out.printf("%d ", counter);
    ++counter;
} while (counter <= 10);
System.out.println("END-"+counter);
```

► Example 2:

```
counter = 100;
do {
    System.out.printf("%d ", counter);
    ++counter;
} while (counter <= 10);
System.out.println("END-"+counter);
```

100 END-101

once

► Example 3:

```
counter = 100;
while (counter <=10) {
    System.out.printf("%d ", counter);
    ++counter;
}
System.out.println("END-"+counter);
```

ZERO

END-100



Exercise 1-3

- ▶ 1) What does the following code print?

```
for (int i = 3; i <= 12; i++){  
    System.out.print(i + " ");  
}
```

- ▶ 2) What will be the value of x after the following code is executed?

```
int x = 10;  
while (x < 100); { //note: there is a ';'   
    x += 10;  
}  
System.out.println(x);
```

- ▶ 3) Which of the following DOES NOT print “#####”?

```
for (int i = 3; i < 9; i++){  
    System.out.print("#");  
}
```

```
int x = -6;  
while (x < 0) {  
    System.out.print("#");  
    x++;  
}
```

```
int y = 1;  
do {  
    System.out.print("#");  
    y++;  
} while (y<=6);
```




2. Branching Statements

The break statement

- ▶ The *break* statement used to terminate the innermost switch, for, while, or do-while statement;
 - ▶ i.e. causes immediate exit from that statement.
- ▶ Execution continues with the first statement after the control statement.
- ▶ Common uses of the break statement are to escape early from a loop or to skip the remainder of a switch.
- ▶ Example:

```
int count;
for (count=1; count<=10; count++) {
    if (count == 5)
        break;
    System.out.printf("%d ", count);
}
System.out.printf("\nBroke out of loop at count=%d\n", count);
```

Loop 10 times

1 2 3 4
Broke out of loop at count=5



2. Branching Statements

The continue statement

- ▶ The *continue* statement skips to the end of the innermost loop's body and evaluates the boolean expression that controls the loop.
- ▶ i.e. skips the remaining statements of **this** iteration in the loop body and proceeds with the **next iteration** of the loop.
- ▶ Example:

1 2 3 4 6 7 8 9 10
Used continue to skip printing 5

```
int count;
for (count=1; count<=10; count++) {
    if (count == 5)
        continue;
    System.out.printf("%d ",count);
}
System.out.printf("\nUsed continue to skip printing 5\n");
```

not

Skipped at 5



2. Branching Statements

for ... ~~continue~~ Vs while ... ~~continue~~

avoid using it

- ▶ In a for statement, the **increment expression** executes, then the program **evaluates** the loop-continuation test.

```
int i = 0, j = 0;
```

```
for (; i < 4; i++){  
    if (i == 2)  
        continue;  
    System.out.printf("%d ", i);  
}  
System.out.printf("%nEND_EXAMPLE1 %d%n", i);
```

```
0 1 3  
END_EXAMPLE1 4
```

- ▶ in while and do...while statements, the program evaluates the loop-continuation test **immediately** after the continue statement executes
 - ▶ if the increment appears **after** the continue statement, the increment statement is not executed. => infinite loop

```
while (j < 4) {  
    if (j == 2)  
        continue;  
    System.out.printf("%d ", j);  
    j++;  
}  
System.out.printf("%nEND_EXAMPLE2 %d%n", j);
```

Increment step is NOT
executed ... infinite loop

```
0 1 ...
```

Exercise 4-5

- ▶ The program calculates the sum of all natural numbers from 1 to the given number using a while loop. Fix the problem.

```
int result=0, i=1;
while (i<=number)
    result += i;
System.out.println("Sum = " + result);
```

- ▶ Rewrite the above program using a for-loop.

```
int result=0;

System.out.println("Sum = " + result);
```



2. Branching Statements

Nested Loops & break

- ▶ It is used to terminate the **innermost** switch, for, while, or do-while statement
- ▶ Example:

```
int row = 3;
for( int i = 0; i < row; i ++ ) {
    for (int j = 0; j < row; j++) {
        if ( i+j >= row )
            break;
        System.out.print("*");
    }
    System.out.println();
}
```

```
***
**
*
```

i	outer	j	inner	i+j	output
0	0<3	0	0<3	0	*
0	0<3	1	1<3		**
0	0>3	2	2<3		***
0	0<3	3	False		-
1	1<3	0	0<3	1	*** *
1	1<3	1	1<3	2	*** **
1	1<3	2	2<3	3=3 break	
2	2<3	0	0<3	2	*** ** *
2	2<3	1	1<3	3=3 break	
3	False				



2. Branching Statements

The return statement

- ▶ The return statement is used to exit from the current method;
 - ▶ The flow of control returns to the statement that follows the original method call.
- ▶ The return statement has two forms:
 - ▶ `return;`
 - ▶ exits the current method
 - ▶ `return variableName;`
 - ▶ returns a value
 - ▶ Note: the data type of the value returned by return must match the type of the method's declared return value.

```
int row = 4;
for (int i= 1; i <= row; i++ ) {
    if( i % 2 == 0)
        return;
    System.out.println("i = " + i );
}
```

i	C	output
1	1<=4	Print "i=1"
	2<=4	Return from the method

i=1



2. Branching Statements

The exit method

- ▶ If you have a program situation where it is pointless to continue execution you can terminate the program with the `System.exit(n)` method
 - ▶ `n` is often used to identify if the program ended normally or abnormally
 - ▶ `n` is conventionally 0 for normal termination and non-zero for abnormal termination

```
if ( i < 3)
    System.exit(0);
```



Exercise 6 – 7 Using nested loops

► Print the following patterns:

```
public static void main(String[] args) {  
    Scanner input = new Scanner(System.in);  
    System.out.println("Enter number of rows: ");  
    int number = input.nextInt();  
}
```

```
11111  
2222  
333  
44  
5
```

```
1  
22  
333  
4444  
55555
```




3. Random Numbers

- ▶ The element of chance can be introduced in a program via an object of class **SecureRandom**
 - ▶ From package `java.security`
 - ▶ Such objects can produce random boolean, byte, float, double, int, long and Gaussian values.
 - ▶ SecureRandom objects produce nondeterministic random numbers that cannot be predicted.
- ▶ The nextInt() that receives an int argument and returns a value from 0 up to, but **not** including, the argument's value.
- ▶ Note: To generate a series of random numbers as a unit, you need to use **a single Random object** - do not create a new Random object for each new random number.

```
SecureRandom randomNumbers = new SecureRandom();  
for (int counter=0; counter<=20; counter++)  
    System.out.printf("%d ", randomNumbers.nextInt(6));
```

0 <= random number <=5

2 4 3 5 1 2 0 0 3 0 3 5 0 2 5 1 2 1 2 0 3



3. Random Numbers Scaling and Shifting

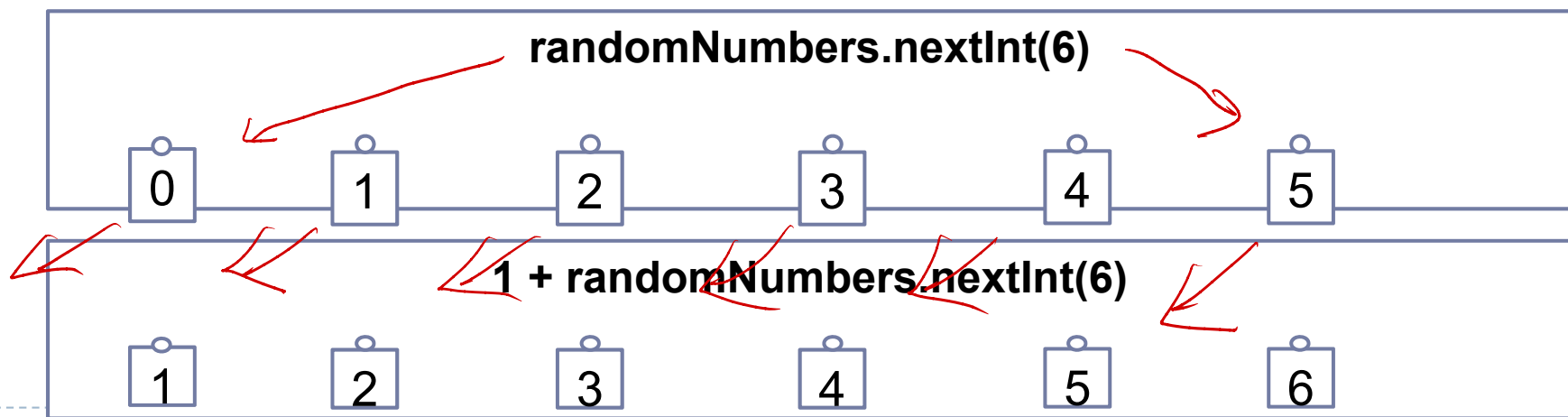
- ▶ Generalized scaling and shifting of random numbers:

```
SecureRandom randomNumbers = new SecureRandom();  
number = shiftingValue + randomNumbers.nextInt(scalingFactor);
```

- ▶ where shiftingValue specifies the first number in the desired range of consecutive integers and scalingFactor specifies how many numbers are in the range

- ▶ Example: Simulating dice

- ▶ Valid values: 1, 2, 3, 4, 5, 6





3. Random Numbers

Random Numbers in CodeRunner

- ▶ We need a random number sequence in CodeRunner.
 - ▶ Therefore we should use: Random class only. For example:

```
import java.util.*;  
...  
Random randObj = new Random(30);  
System.out.println(randObj.nextInt(10));  
System.out.println(randObj.nextInt(10));  
System.out.println(randObj.nextInt(10));
```



6
8
5

0 <= random number <=9

- ▶ SecureRandom Vs Random
 - ▶ The basic and important difference between both is SecureRandom generate more non predictable random numbers



Exercise 8

- ▶ Prompts the user to guess a number (from 0 to 9, inclusive)
- ▶ Should generate a random number between the above range

```
int guessNum, randNum;  
  
if (guessNum == randNum)  
    System.out.printf("You won.");  
else {  
    System.out.printf("You lost. My number is %d%n.", randNum);  
}
```

Enter your guess number (0-10):

5

You lost. My number is 9

Enter your guess number (0-10):

5

You lost. My number is

Exercise 9

- ▶ Prompts the user to enter two integers
- ▶ Prints the average of all even integers between firstNumber and lastNumber (inclusive)

```
int total = 0, count=0;
```

Exercise 10

- ▶ Complete the code which reads **THREE** rows of numbers from the User. Enter -1 to end each row.

```
System.out.println("Enter 3 rows of digits('-1' to end each row):");
for (int i=0; i < 3; i++) {
    num = input.nextInt();
    while (
        ) {
        rows = rows + num + " ";

    }
    rows += "\n";
}
System.out.println("\nRows of digits\n" + rows);
```

4
6
7
-1
2
3
-1
1
4
5
-1

Rows of digits

4 6 7
2 3
1 4 5